

Learning Strategy Neural Network

Afiahayati
Magister Kecerdasan Artifisial
Departemen Ilmu Komputer dan Elektronika
FMIPA Universitas Gadjah Mada
afia@ugm.ac.id



Agenda (Update)

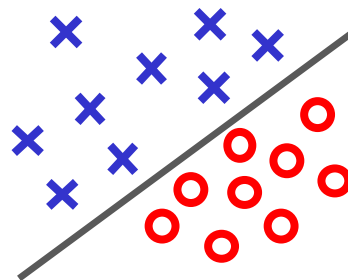
- Meeting 1 : Perkenalan, RPKPS, Linear Classifier (24 Aug) – offline
- Meeting 2 : ANN – SLP and MLP – **submit Assignment 1** (7 Sept) – offline
- Meeting 3 : Discussion Assignment 1 (14 Sept) – offline
- Meeting 4 : Guest Lecturer (19 Sept) – online
- Meeting 5 : Learning Strategy (21 Sept) – offline
- Meeting 6 : Introduction to Deep Learning + CNN + Assignment 2 (28 Sept) – online
- **Midterm Examination**
- Meeting 7 : Discussion and Presentation - **Mini Project** - (16 Oct) - offline

Linear and Non-Linear Problems

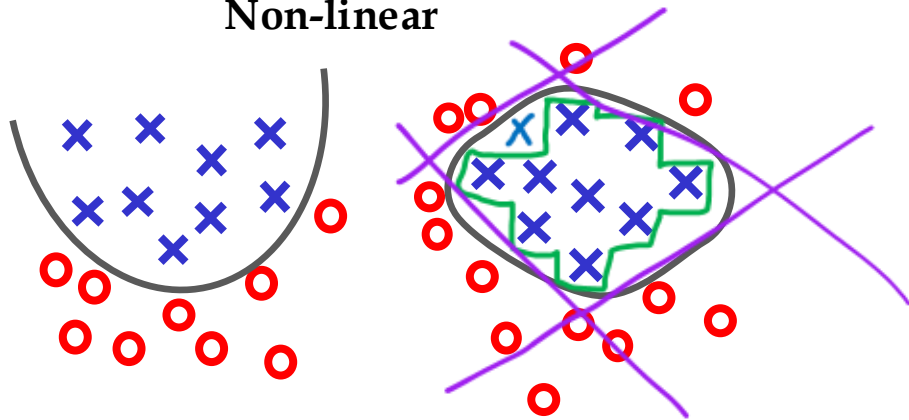
Classification problems can be categorized as:

- Linear problems, for example the OR and AND functions
- Non-Linear problems, for example the XOR function

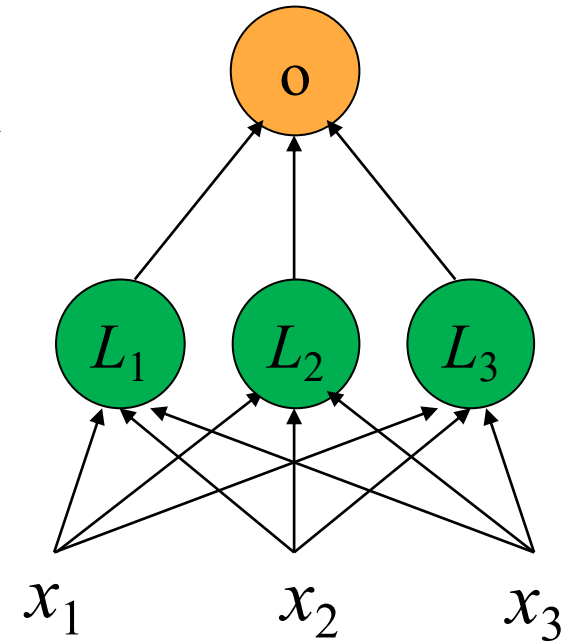
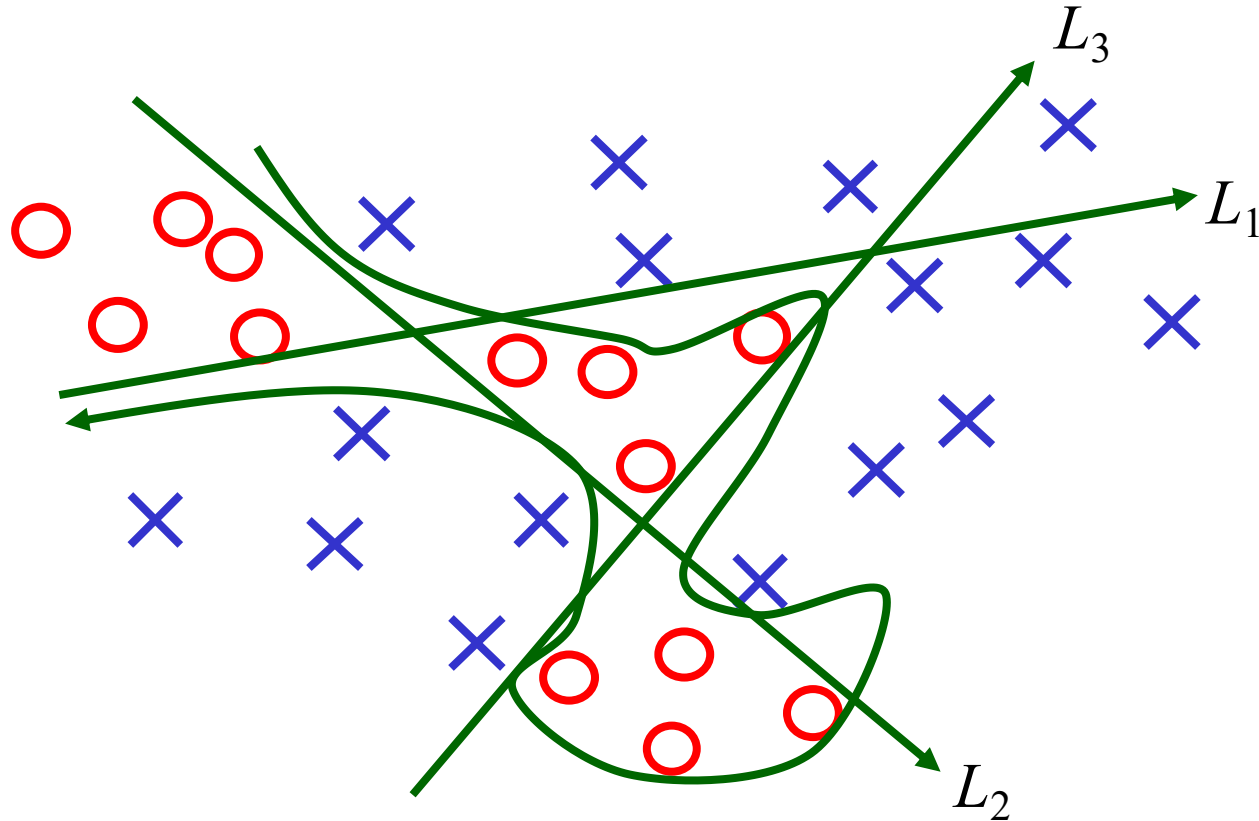
Linear



Non-linear

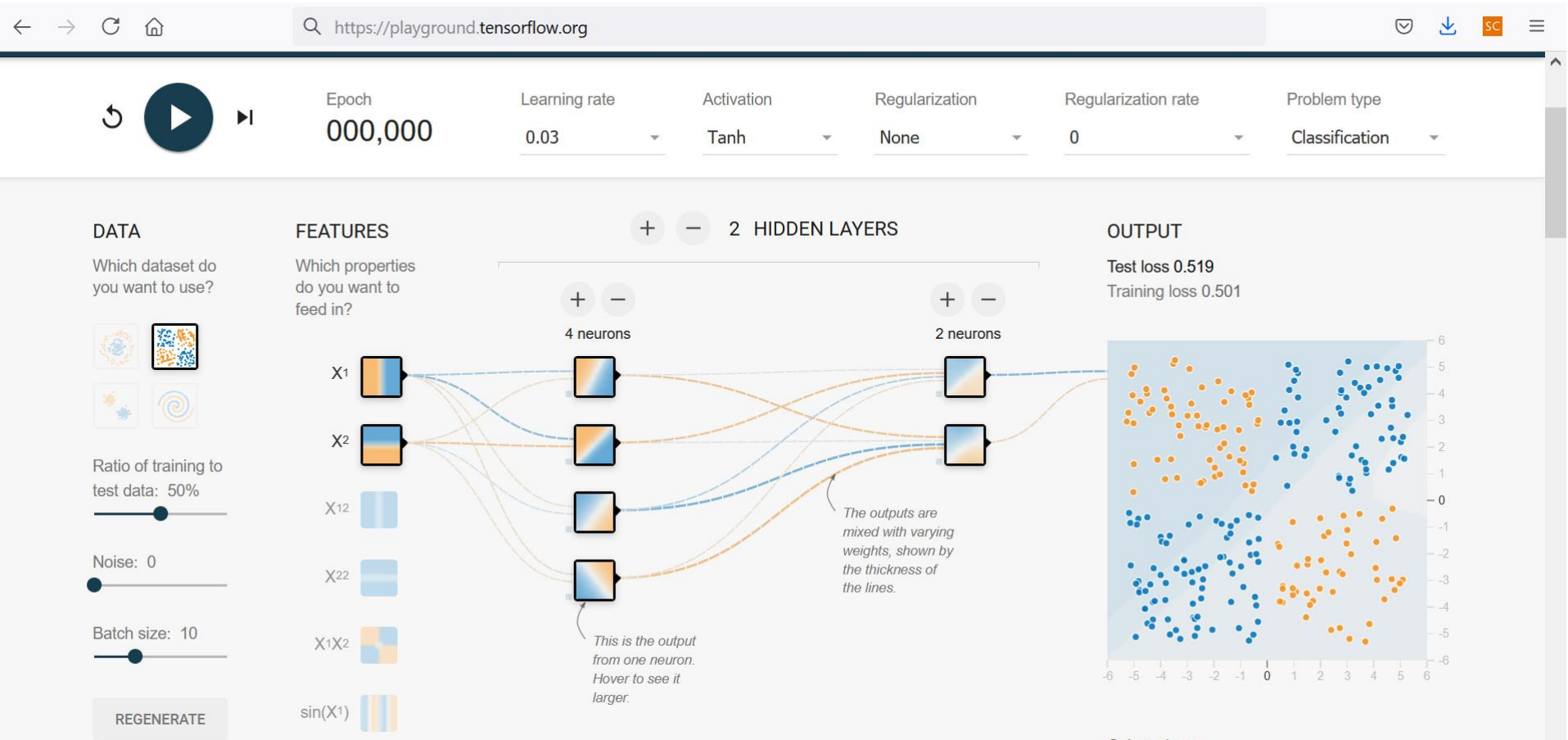


For non-linear problems and more complex problems, a **Multi-layer ANN architecture is used**



ANN Architecture Design

<https://playground.tensorflow.org/>



ANN Stages: Learning (*Learning*)

1. Initialize weights W
2. Update the weights so that the ANN output is consistent with the class label in the training data

a. Objective function (loss function) $J(W) = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - f(x^{(i)}; W) \right)^2$

y – target

f – prediction

- b. Find new weights by minimizing the objective function, for example:
the *backpropagation algorithm*

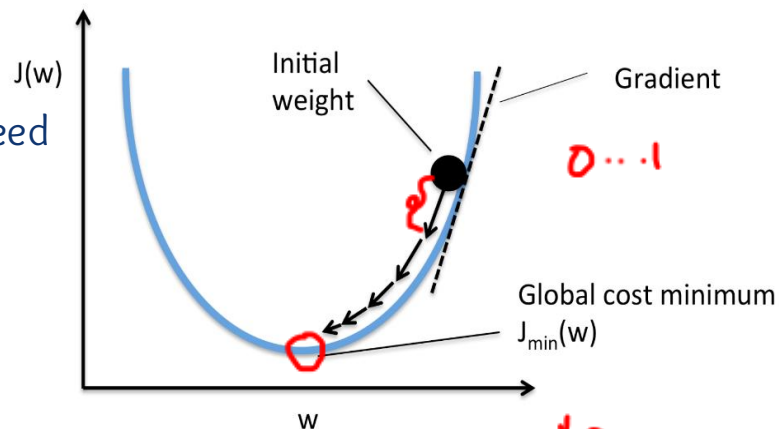
Learning Algorithm (Learning)

To design a learning algorithm, there are several things that need to be considered:

1. Stopping criteria? **convergence, iterations (epoch)**
2. What direction? **gradient descent**
3. How many (steps) are needed? **the learning rate value (η)**

Backpropagation Gradient Descent Algorithm

- Initialize weights randomly
- Iterate until convergence or maximum iterations
 - Calculate the gradient, $\frac{\partial J(W)}{\partial W}$
 - Update the weights, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
- Return the weight values



$$W^* = \operatorname{argmin}_W J(W)$$

$$J(W) = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - f(x^{(i)}; W) \right)^2$$

If the *learning rate* value is too small, it takes longer to converge; if it is too large, it makes the model unstable

Learning Algorithm (*Learning*)

Stochastic Gradient Descent Algorithm

GD

- Initialize weights randomly
- Iterate until convergence or maximum iterations
 - Read **each** data point i
 - Calculate the gradient, $\frac{\partial J_i(W)}{\partial W}$
 - Update the weights, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
- Return the weight values



SGD

- Initialize weights randomly
- Iterate until convergence or maximum iterations

- Read **batch B** of data points

- Calculate the gradient, $\frac{\partial J(W)}{\partial W} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(W)}{\partial W}$

- Update the weights, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$

- Return the weight values



Stages of Multi-layer Perceptron ANN Learning

Step 0 – Initialize weights, learning rate, maximum iterations

Step 1 – Read input vector X

Step 2 – Perform iteration (*epoch*)

Step 3 – Calculate the output of neurons in the hidden layer and output layer

Step 4 – Calculate the *back propagate error* (at the output layer and hidden layer)

Step 5 – Update all weights (at the output layer and hidden layer)

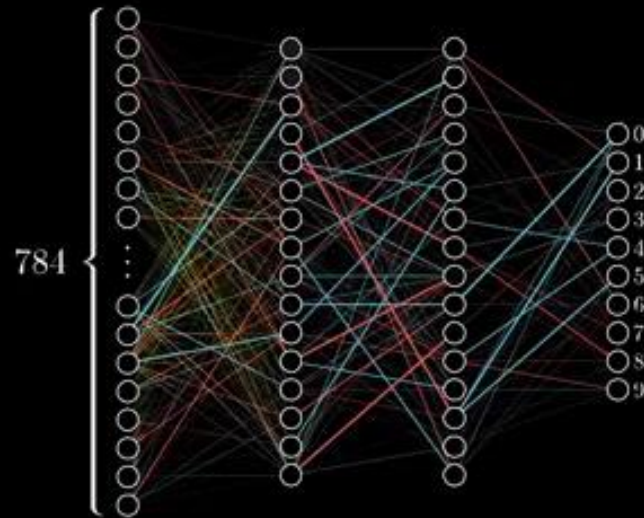
Step 6 – Repeat steps 3 – 5 until the weights converge or reach the maximum iterations

Step 7 – Output the weight matrix (at the output layer and hidden layer)

<https://machinelearningmastery.com/implement-backpropagation-algorithm-scratch-python/>

Example of the ANN Learning Process ANN

Training in
progress...



Tools / Lab Online

- TensorFlow is an end-to-end open-source platform for machine learning
- Keras is a deep learning API written in Python, running on top of the machine learning platform TensorFlow. It was developed with a focus on enabling fast experimentation. Being able to go from idea to result as fast as possible is key to doing good research.



<https://www.tensorflow.org/overview/>

https://keras.io/getting_started/

<https://keras.io/examples/>

Tools / Lab Online

<https://keras.io/api/models/>

https://keras.io/guides/sequential_model/

<https://keras.io/api/layers/>

- Keras Model Sequential and Layers

```
model = keras.Sequential(  
    [  
        layers.Dense(64, activation="relu", name="layer1"),  
        layers.Dense(32, activation="relu", name="layer2"),  
        layers.Dense(4, name="layer3"),  
    ]  
)
```

or

```
model = keras.Sequential()  
model.add(layers.Dense(64, activation="relu"))  
model.add(layers.Dense(32, activation="relu"))  
model.add(layers.Dense(4))
```

Example of ANN implementation using the keras library

- Load data

```
from sklearn import datasets
```

```
from sklearn.model_selection import train_test_split
```

```
from keras.utils import to_categorical
```

```
iris = datasets.load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size=.10)
```

```
X_train, X_val, Y_train, Y_val = train_test_split(X_train, Y_train, test_size=.2)
```

```
print('X_train', X_train.shape)
```

```
print('X_val', X_val.shape)
```

```
print('X_test', X_test.shape)
```

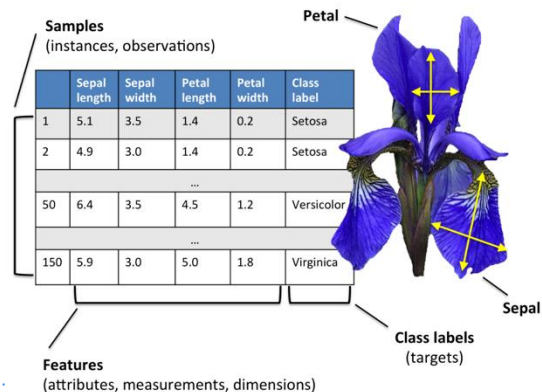
```
Y_train = to_categorical(Y_train,3)
```

```
Y_val = to_categorical(Y_val,3)
```

```
Y_test = to_categorical(Y_test,3)
```

150 → 10% → 15

135 → 20%



Example of ANN implementation using the keras library

- Define model and compile model

```
from keras.models import Sequential
from keras.layers import Flatten, Dense

model = Sequential()
model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dense(3, activation='softmax'))

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['acc'])
```

↓
under

Example of ANN implementation using the keras library

- Fit model

```
model.fit(X_train,Y_train,epochs=64,batch_size=5,validation_data=(X_val,Y_val))  
model.summary()
```

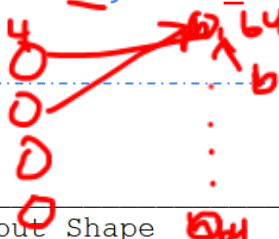
Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 4)	0
dense (Dense)	(None, 64)	320
dense_1 (Dense)	(None, 3)	195

Total params: 515

Trainable params: 515

Non-trainable params: 0



$(4+1) \times 64 = 320$
 $64 \times 3 + 3 = 195$

Example of ANN implementation using the keras library

- Evaluation model

```
loss, accuracy = model.evaluate(X_test, Y_test)
print('Akurasi Testing MLP:', accuracy)
```

```
1/1 [=====] - 0s 0s/step - loss: 0.0393 - acc: 1.0000
Akurasi Testing ANN: 1.0
```


Example implementation of a *fully connected layer* architecture for digit recognition

The MNIST *Handwritten Digit* dataset is divided into 3:

- 55,000 training data
- 10,000 test data
- 5,000 validation data

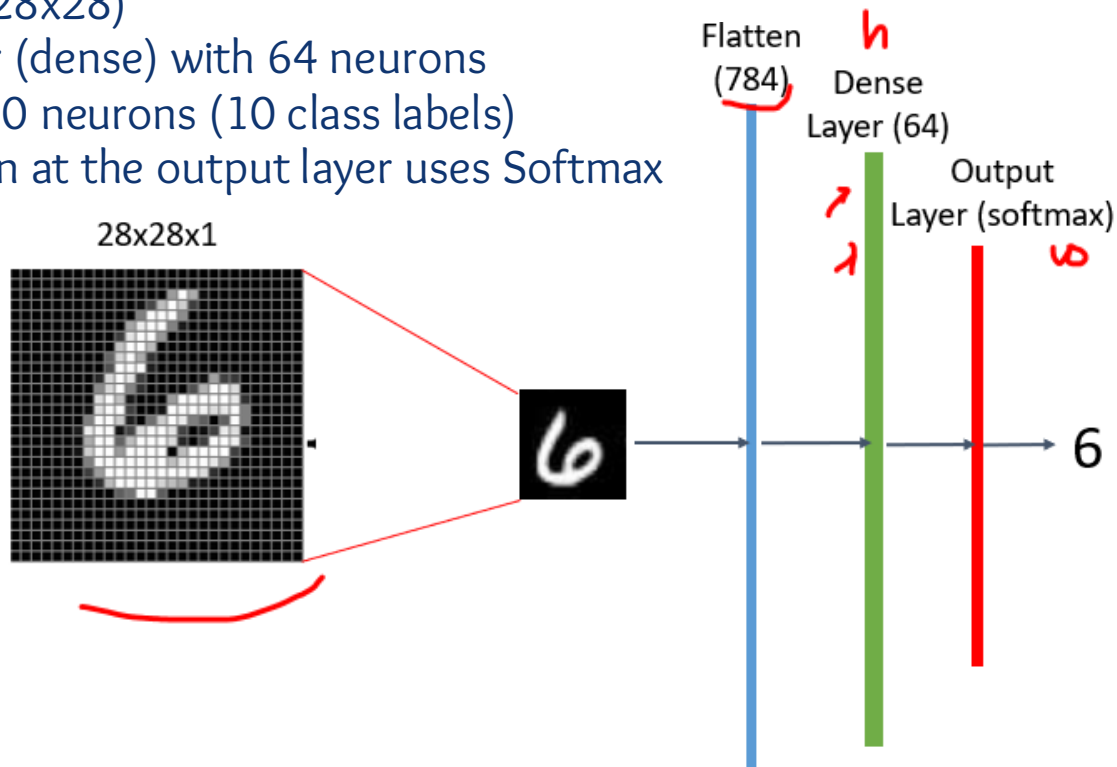
Each image is 28×28 pixels in size, and the class label is converted to *one hot encoded*

label = 5 	label = 0 	label = 4 	label = 1 	label = 9 	0	[1 0 0 0 0 0 0 0 0 0]
label = 2 	label = 1 	label = 3 	label = 1 	label = 4 	1	[0 1 0 0 0 0 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	2	[0 0 1 0 0 0 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	3	[0 0 0 1 0 0 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	4	[0 0 0 0 1 0 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	5	[0 0 0 0 0 1 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	6	[0 0 0 0 0 0 1 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	7	[0 0 0 0 0 0 0 1 0 0]

Example implementation of a *fully connected layer* architecture for digit recognition

The ANN architecture used for digit recognition:

- Input layer size 784 (28x28)
- There is 1 hidden layer (dense) with 64 neurons
- The output layer has 10 neurons (10 class labels)
- The activation function at the output layer uses Softmax



Example implementation of a *fully connected layer* architecture for digit recognition

1. Load data

```
import keras
from keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train.reshape(-1, 28,28,1)
X_test = X_test.reshape(-1, 28,28,1)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
X_train /= 255
X_test /= 255
y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)
```

Example implementation of a *fully connected layer* architecture for digit recognition

2. Define Model

```
from keras.models import Sequential
from keras.layers import Flatten, Dense

model1 = Sequential()
model1.add(Flatten())
model1.add(Dense(64,activation='relu'))
model1.add(Dense(10,activation='softmax'))
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====	=====	=====
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 64)	✓ 50240
dense_1 (Dense)	(None, 10)	✓ 650
=====	=====	=====

Total params: 50,890 ✓

Trainable params: 50,890 ✓

Non-trainable params: 0

Example implementation of a *fully connected layer* architecture for digit recognition

3. Compile Model, Fit Model, Save Model, and Evaluate Model

```
model1.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['acc'])  
history =  
model1.fit(X_train,y_train,epochs=10,batch_size=100,validation_data=(X_test,y_test))  
model1.save('my_model1.h5')  
model1.evaluate(X_test,y_test)
```

```
313/313 [=====] - 0s 1ms/step - loss: 0.0839 - acc: 0.9761
```

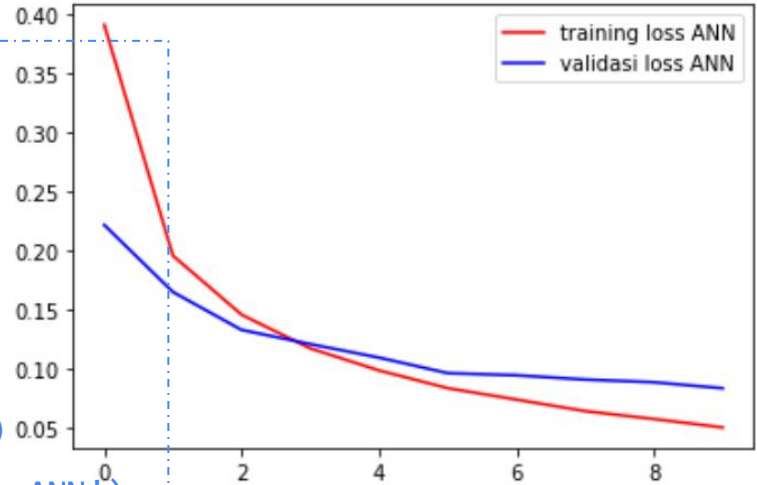
```
[0.08388985693454742, 0.9761000275611877]
```

Example implementation of a *fully connected layer* architecture for digit recognition

3. Visualize Model Evaluation

```
import matplotlib.pyplot as plt

epochs = range(10)
loss1 = history1.history['loss']
val_loss1 = history1.history['val_loss']
plt.plot(epochs, loss1, 'r', label='training loss ANN')
plt.plot(epochs, val_loss1, 'b', label='validation loss ANN')
plt.legend()
```



Example implementation of a *fully connected layer* architecture for digit recognition

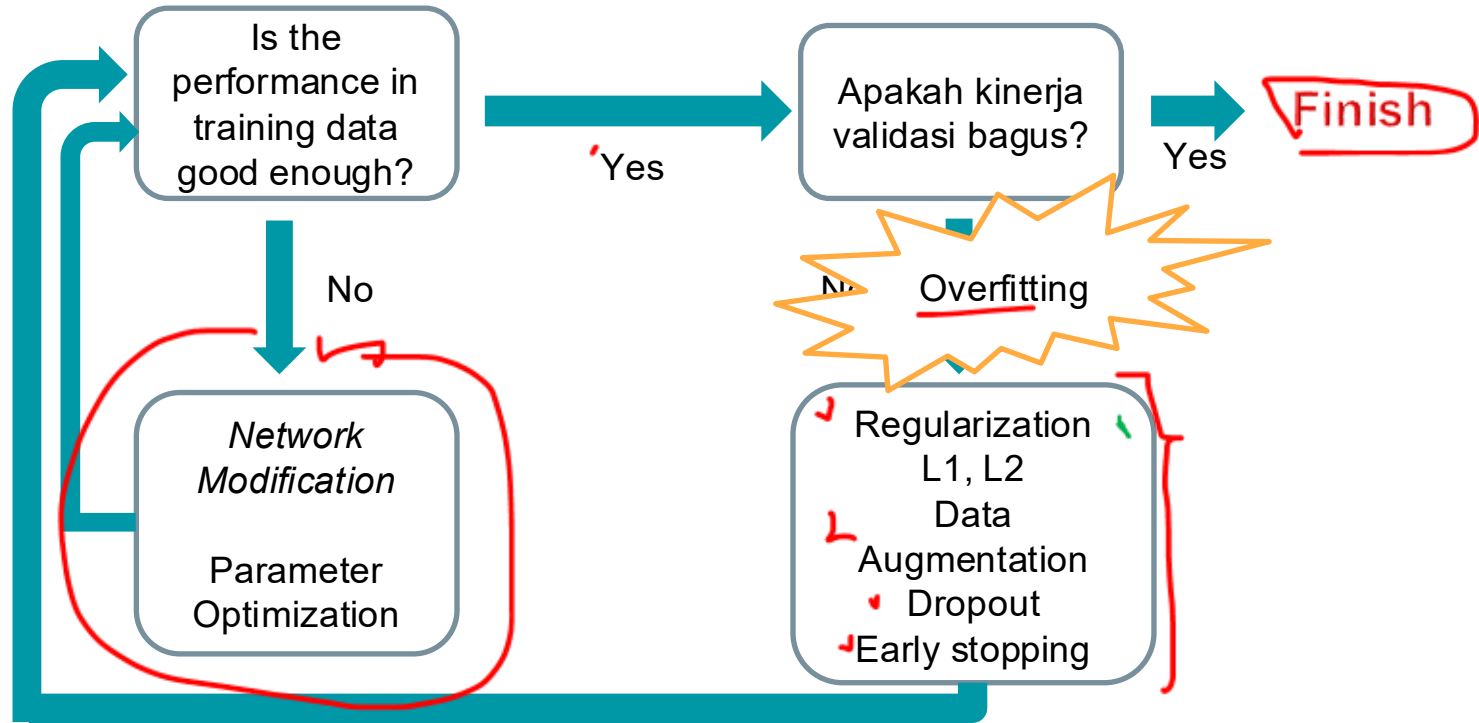
3. Load Model and Prediction

```
import numpy as np
from keras.models import load_model

model_simpan = load_model('my_model.h5')
pred = model_simpan.predict(X_test)
print('label actual:', np.argmax(y_test[30]))
print('label prediction:', np.argmax(pred[30]))
```

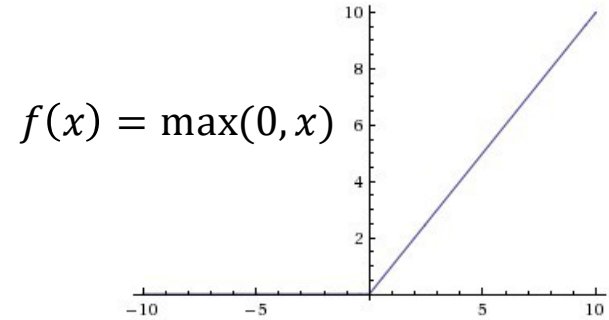
```
label actual: 3
label prediction: 3
```

Learning Strategy



Network Modification

- Modify the NN Architecture : add hidden layer, add neurons or changes to others; architecture
- Change the activation function : tanh, relu, ...



Rectified Linear Unit function (ReLU)

Optimasi parameter

The learning rate value influences the calculation of new weights; generally, the use of a learning rate that adjusts based on the gradient (adaptive learning rate) yields better model performance. Examples of adaptive learning rate algorithms include:

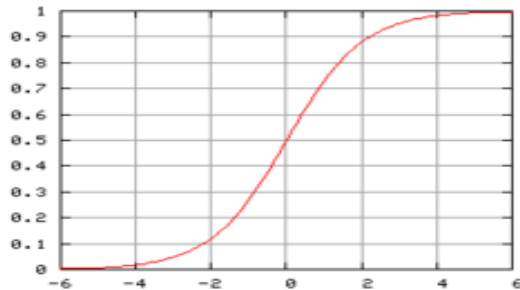
- Adagrad [John Duchi, JMLR 2011]
- Adadelta [Matthew D. Zeiler, arXiv 2012]
- Adam [Diederik P. Kingma, ICLR 2015]
- AdaSecant [Caglar Gulcehre, arXiv 2014]
- RMSprop <https://www.youtube.com/watch?v=O3sxAc4hxZU>

Activation Function

- The activation function turns the neuron into a non-linear one
- Some examples of activation functions commonly used in the ANN method are as follows

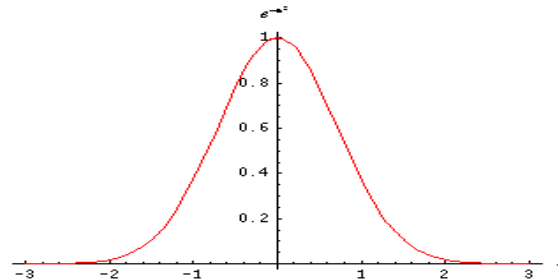
Sigmoid function

$$a(f) = \frac{1}{1 + \exp(-f)}$$



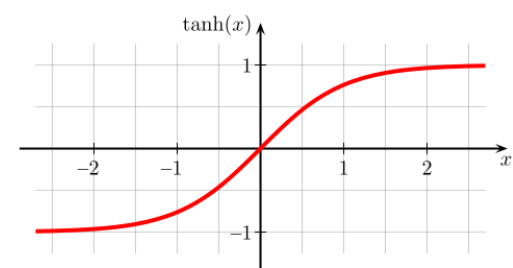
Gaussian function

$$a(f) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2}\left(\frac{f-\mu}{\sigma}\right)^2\right)$$



Tangent Hyperbolic function

$$\tanh(x) = 2\sigma(2x) - 1$$



Preventing Overfitting

Regularization

Regularization is employed to reduce generalization error by preventing the model from becoming overly complex. It is implemented by adding a regularization term—applied to all parameters (weights)—to the objective function.

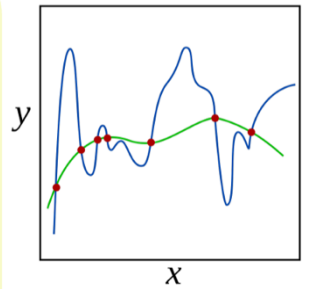
.

- Regularization L1 norm
 - Adding the sum of the absolute weights as a penalty term to the objective function
- Regularization L2 norm (*weight decay*)
 - Adding the sum of the absolute weights as a penalty term to the objective function



$$\text{Error} = \frac{1}{N} \sum (\tau - p) + \alpha \sum |\beta|$$

L1 Regularization or LASSO

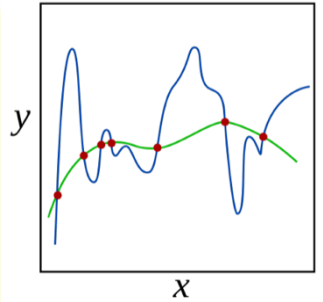


- L1 Regularization technique is also known as **LASSO** or Least Absolute Shrinkage and Selection Operator. In this, the penalty term added to the cost function is the summation of absolute values of the coefficients. Since the absolute value of the coefficients is used, it can reduce (or shrink) the coefficient to 0 and such features may completely get discarded in LASSO. Thus, we can say, LASSO helps in Regularization as well as Feature Selection.
- Following is the equation of Cost function with **L1 penalty** term:

$$\mathcal{E}_{\text{mse}} = \frac{1}{N} \sum_{t=1}^N (Y_t - (\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n))^2 + \alpha \sum_{t=0}^n |\beta_t|$$

- Here, **alpha** is the multiplier term.

L2 Regularization or Ridge



- L2 Regularization technique is also known as **Ridge**. In this, the penalty term added to the cost function is the summation of the squared value of coefficients. Unlike the LASSO term, the Ridge term uses squared values of the coefficient and can reduce the coefficient value near to 0 but not exactly 0. Ridge distributes the coefficient value across all the features.
- Following is the equation of Cost function with **L2 penalty** term:

$$\epsilon_{\text{mse}} = \frac{1}{N} \sum_{i=1}^N (Y_i - (\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n))^2 + \alpha \sum_{i=1}^n \beta_i^2$$

- Here, **alpha** is the multiplier term. $0.5^2 = 0.25$

Preventing Overfitting

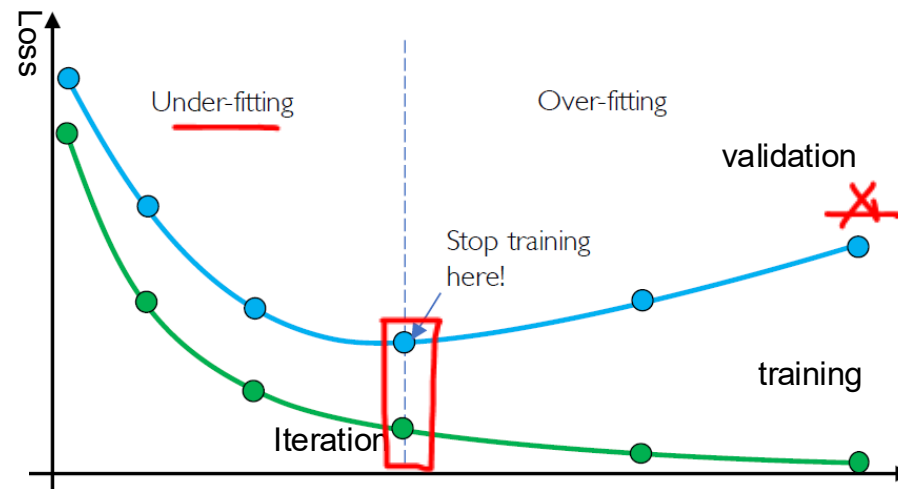
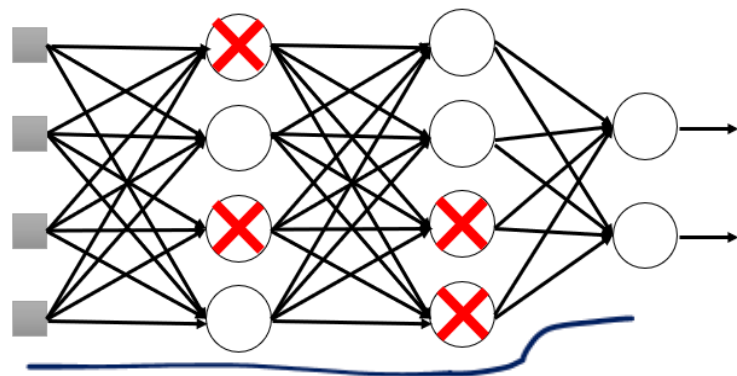
How to tune parameters to avoid overfitting, thereby making the model more generalizable.

- Dropout

- Determination of neurons to be deactivated based on the dropout percentage p% 50%

- Early stopping

- Training iterations are stopped if the generalization error begins to rise.



Preventing Overfitting

Adding Training Data (Data Augmentation)

- The process of increasing the variety of training data so that the resulting model performs better at predicting test data
- The data augmentation method used depends on the type of input data
- Oversampling methods for numerical data: SMOTE, ADASYN, etc.
- Examples of image data augmentation: rotation, translation, flipping, and zooming



~~60~~ →
~~imbalanced~~ →
1000 → ✓
200 → ✓

text data → keren bgt guys

Referensi

- 虞台文, Feed-Forward Neural Networks, Course slides presentation
- Andrew Ng, Machine Learning, Course slides presentation
- Michael Negnevitsky, Artificial Intelligence : A Guide to Intelligent Systems, Second Edition, Addison Wesley, 2005.
- Hung-yi Lee, Deep Learning Tutorial
- Alexander Amini, Intro to Deep Learning, MIT 6.S191, 2021